# ECSE 429 - Project Part 3 Report

**by David Vo(261170038) and David Zhou (261135446)**

## 1. OVERVIEW

This part of the project focuses on **non-functional testing** of the Todo REST API. Unlike functional testing, which verifies correctness of behavior, non-functional testing evaluates system qualities such as:

- Performance
- Scalability
- Reliability
- Maintainability and code quality

Two complementary approaches were used:

1. **Dynamic performance testing** using a custom JUnit performance test suite and system monitoring.
2. **Static analysis** using **SonarQube Community Edition** to evaluate code quality, complexity, and technical debt.

The objective was to measure how the system behaves under increasing load and to identify design or implementation risks before deployment.

## 2. DELIVERABLE SUMMARIES

The following deliverables were produced:

- Performance test suite video
- Written report

## 3. DYNAMIC TEST SUITE IMPLEMENTATION

### 3.1. Test Environment

- Java JUnit 5 test framework
- java.net.http.HttpClient
- Local REST server: `http://localhost:4567`
- OS-level monitoring via shell script
- Maven Surefire for execution

The test suite executes real HTTP requests against the running REST service.

## 3.2. Test Design

The performance test suite was implemented in `TodoPerformanceTest.java` and organized into the following categories:

**Create performance tests**

- 50, 100, 200, 500, 1000 objects

**Update performance tests**

- 50, 100, 200, 500 objects

**Delete performance tests**

- 50, 100, 200, 500 objects

**Comprehensive scalability test**

- 10 → 1000 objects
- Measures create, update, and delete in sequence

Each test:

- Resets system state
- Uses **randomized realistic data**
- Measures:
  - Total execution time
  - Average time per object
- Collects JVM memory statistics
- Runs concurrently with OS monitoring

## 3.3. Measure Metrics

| Category | Description |
|---|---|
| Response time | Total and average operation time |
| Scalability | Growth behavior from 10 to 1000 objects |
| CPU usage | System load under stress |

| | |
|---|---|
| Memory usage | Free memory evolution |
| JVM memory | Heap behavior |

# 4. Performance Results Summary

## 4.1. API Transaction Performance

Observed averages:

| Operation | Typical Avg Time |
|---|---|
| Create | 0.88 – 1.79 ms/object |
| Update | 0.96 – 3.20 ms/object |
| Delete | 0.58 – 1.17 ms/object |

Key observations:

- Create operations scale efficiently up to 1000 objects.
- Update operations show the highest variability (0.96–3.20 ms/obj), likely due to JVM warmup at low object counts, but total time scales linearly.
- Delete operations are consistently fast.
- No test failures, crashes, or service unavailability occurred.

The system demonstrates **linear scalability** with no major degradation.

## 4.2. CPU Utilization

CPU usage peaked above **400%**, indicating:

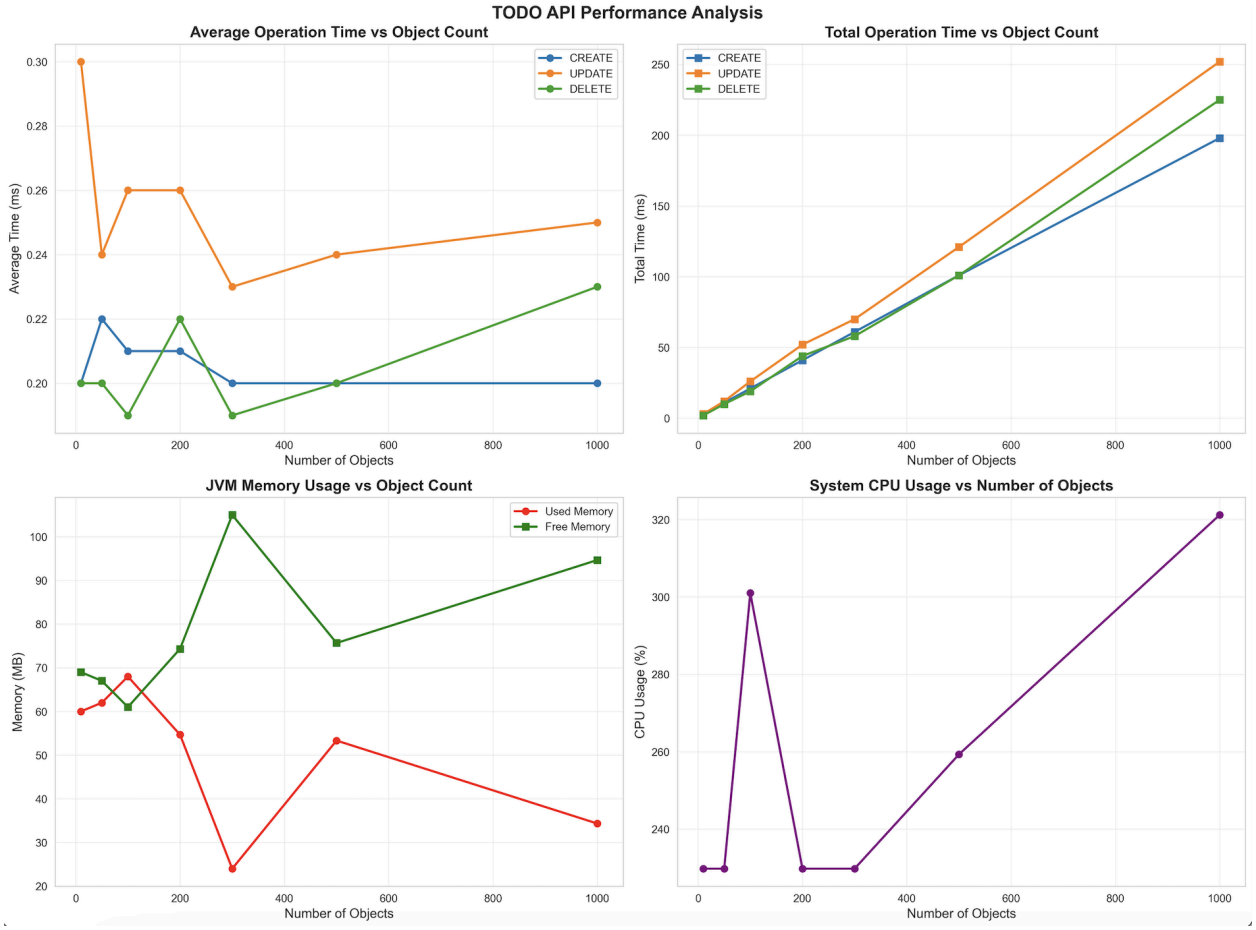
- Heavy multi-core utilization
- REST server and tests actively consuming resources
- No CPU starvation or system stall
- As shown in the CPU vs number of objects chart, CPU usage increases with object count, peaking at approximately 456% at higher loads, consistent with the REST server processing larger batches of requests

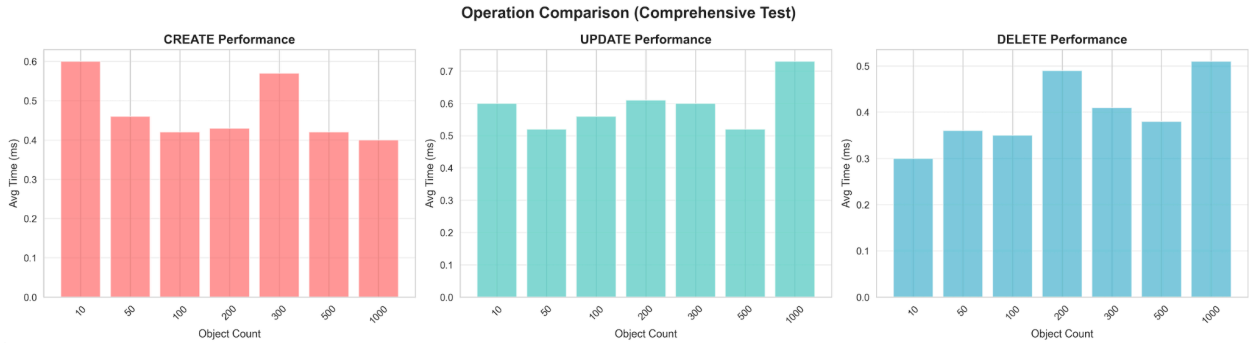
## 4.3. Memory Behaviour

Memory remained bounded and stable:

- JVM heap fluctuated between ~20 MB and ~81 MB.
- OS free memory dropped under heavy load but recovered.
- No memory leak patterns were observed.

## 4.4. Charts




# 5. OBSERVED PERFORMANCE RISK

### High CPU saturation

- CPU exceeded 400% during peak tests.
- Under real concurrent users, this may cause:
    - Increased response latency
    - Thread contention

### Lack of concurrency testing

- Current tests are sequential.
- Real systems face simultaneous users.

### No persistence-layer stress

- Tests do not isolate database behavior.
- Disk-bound deployments may behave differently.

## 6. PERFORMANCE-BASED RECOMMENDATIONS

### Implement asynchronous handling

Use non-blocking IO or thread pools for improved throughput.

### Add concurrent user tests

Introduce tools such as:

- JMeter
- Gatling
- Locust

### Introduce caching

Frequently accessed resources could be cached.

### Add request throttling

Prevent CPU exhaustion under burst traffic.

### Monitor memory growth long-term

Add long-running soak tests.

# 7. STATIC ANALYSIS WITH SONARQUBE

## 7.1. Tools and Setup

- SonarQube Community Edition
- Analysis performed on: https://github.com/eviltester/thingifier

SonarQube was used to detect:

- Bugs
- Code smells
- Technical debt
- Code duplication
- Security issues
- Complexity hotspots

## 7.2 SonarQube Results Summary

| Metric | Result | Interpretation |
|---|---|---|
| Bugs | 15 (D) | High priority |
| Vulnerabilities | 0 (A) | Excellent |
| Security hotspots | 8 (E) | Need manual review |
| Code smells | ~1600 (A) | Maintainable |
| Technical debt | ~16 days | Acceptable |
| Duplications | 0.3% | Excellent |
| Coverage | 0% | Missing metrics |

## 7.3 Complexity and Maintainability

Findings:

- Some methods show high cyclomatic and cognitive complexity.

- Several long methods and duplicated logic blocks exist.
- Maintainability rating A indicates low debt density.

Complex methods increase:

- Defect probability
- Testing difficulty
- Refactoring cost

## 8. STATIC ANALYSIS RECOMMENDATIONS

### Bug resolution (highest priority)

Fix all Blocker and Critical bugs first.

### Reduce complex methods

- Break large methods into smaller units.
- Apply Single Responsibility Principle.

### Review all security hotspots

- Validate authentication paths
- Verify external inputs
- Review file and request handling

### Refactor repeated patterns

Even with low duplication, repeated logic increases maintenance cost.

## 9. CONCLUSION

The Todo REST API demonstrates:

Strong performance scalability
Stable memory behavior
No crash or degradation up to 1000 objects
Excellent duplication and maintainability ratings
No security vulnerabilities

However, important risks were identified:

High CPU load under stress
15 detected bugs

No code coverage metrics
Complex and long methods
Security hotspots unreviewed

By applying the recommended improvements, the system can significantly enhance its:

- Scalability robustness
- Reliability
- Maintainability
- Security assurance